

Reviewer Pack — Minimal Geometric Complexity and DPLL Proof Tree Height

Entry a6f2ec052ae4 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-07 06:06:18 UTC

Minimal Geometric Complexity and DPLL Proof Tree Height

Entry ID: a6f2ec052ae4 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-07 06:06:18 UTC

1. Conjecture

Field A (mathematical branch): Geometric Group Theory (Hyperbolic Geometry) **Field B** (complexity object): Boolean Satisfiability (DPLL Proof Complexity)

Statement:

For every Boolean satisfiability problem, the height of its minimal DPLL proof tree is upper-bounded by a polynomial in the geometric complexity of the corresponding hyperbolic embedding of the clause set.

Rationale (proposer's reasoning):

Geometric group theory provides a framework for studying symmetries and distances in discrete structures. The geometric complexity of a structure can provide insights into its inherent complexity. In this conjecture, we propose that the height of the DPLL proof tree is related to the geometric properties of the clause set's hyperbolic embedding, suggesting that certain geometric constraints could limit the proof length.

Taxonomy category: GeometricGroupTheory (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 5f8d9e3010efa0c9

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the mean height of the minimal DPLL proof trees for all tested Boolean satisfiability problems is less than or equal to a polynomial multiple (with constant coefficient ≤ 10) of the geometric complexity, as calculated using hyperbolic embedding. The criterion is falsified if any single instance has a DPLL proof tree height exceeding this polynomial bound.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "DPLL proof tree height" AND "geometric group theory" - "Boolean satisfiability problem" AND "hyperbolic geometry embedding" - "polynomial bound" AND "DPLL proof complexity" AND "hyperbolic geometry"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_clauses(n):
        clauses = []
        for _ in range(n):
            clause = {random.randint(1, n) for _ in range(random.randint(1, 3))}
            clauses.append(clause)
        return clauses

    def dpll_proof_tree_height(clause_set):
        if not clause_set:
            return 0
        variables = set()
        for clause in clause_set:
            variables.update(clause)
        true_clauses = {c for c in clause_set if any(var in c for var in variables)}
        false_clauses = clause_set - true_clauses
        if not true_clauses or not false_clauses:
            return 1
        height = 0
        for variable in variables:
            new_true_clauses = {c for c in true_clauses if variable in c}
            new_false_clauses = {c for c in false_clauses if variable not in c}
            height = max(height, dpll_proof_tree_height(new_true_clauses) + 1, dpll_proof_tree_height(new_false_clauses) + 1)
        return height

    def geometric_complexity(clause_set):
        n = len(clause_set)
```

```

        total_length = sum(len(c) for c in clause_set)
        return Fraction(total_length, n)

max_n = 0
instances_tested = 0
total_height = 0
conjecture_holds = True
counterexample = ""

for n in [5, 10, 15, 20, 30, 40]:
    if n > max_n:
        max_n = n

    for _ in range(5): # Ensure at least 30 instances per seed
        clauses = generate_clauses(n)
        height = dpll_proof_tree_height(clauses)
        complexity = geometric_complexity(clauses)

        total_height += height
        instances_tested += 1

        if height > 10 * complexity:
            conjecture_holds = False
            counterexample = f"n={n}, height={height}, complexity={complexity}"

mean_height = Fraction(total_height, instances_tested)
support_fraction = Fraction(instances_tested - (instances_tested if not conjecture_holds else

return {
    "metric_name": "DPLL Proof Tree Height",
    "metric_value": float(mean_height),
    "instances_tested": instances_tested,
    "n_max": max_n,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43,

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_height = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_height} std=0.0 support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample={result['counterexample']} first_failing_seed={first_failing_seed}")
    else:

```

```
print("RESULT: INCONCLUSIVE reason=unknown")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_48a86540.py", line 90, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_48a86540.py", line 62, in run_trial
    height = dpll_proof_tree_height(clauses)
             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_48a86540.py", line 34, in dpll_proof_tr
    true_clauses = {c for c in clause_set if any(var in c for var in variables)}
                   ^
```

TypeError: unhashable type: 'set'

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from verifying the conjecture's support or falsification. | next: Investigate and fix the error in the test code to allow for proper execution and verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13483
2	propose	ollama_remote	glm4:latest	0	0	12681
3	preregistration	ollama_remote	glm4:latest	0	0	9293
4	novelty	ollama_remote	glm4:latest	0	0	10965
5	novelty	ollama_remote	glm4:latest	0	0	8651
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	28515
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11031
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11567
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11225
10	judge	ollama_remote	glm4:latest	0	0	20529

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 137939 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/a6f2ec052ae4.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/a6f2ec052ae4.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/a6f2ec052ae4.tar.gz (if generated)